Zonoth

SESSION 10 · 2H · CAPSTONE

Ship Pinboard like production

Thirty minutes on what "production-ready" actually means and how real teams package it — then eighty minutes building it yourself, from a blank folder.

Production checklist | Helm & Kustomize | GitOps | Managed Kubernetes | Capstone lab

How today runs

1 · Production-ready (30 min)

The checklist every workload owes the platform: probes, resources, security context, PDB, HPA, config and secrets, labels, namespace guardrails.

2 · Packaging & delivery

Helm charts, values and releases · Kustomize bases and overlays · GitOps with Argo CD or Flux · what changes on a managed cluster.

3 · Demo · then a 10 min break

`helm install ingress-nginx` and `kubectl apply -k overlays/dev`, side by side on the same cluster.

4 · Capstone (80 min)

Deploy the whole of Pinboard into `pinboard-prod` from an empty folder and score yourself with `verify.sh`.

01

The production-ready checklist

Nine sessions of objects, arranged into the list you actually run down before a release.

THE LIST

What "production-ready" means

Health

Startup, readiness and liveness probes. Without readiness, every rolling update is a small outage.

Resources

Honest `requests` so the scheduler can do its job, plus a memory `limit` so one Pod can't take the node.

Security

Non-root, read-only root filesystem, all capabilities dropped — inside a `restricted` namespace.

Availability

More than one replica, a PodDisruptionBudget for drains, an HPA for load.

Config & secrets

Nothing baked into the image: ConfigMap for settings, Secret for credentials.

Identity

`app.kubernetes.io/*` labels, its own namespace, ResourceQuota and LimitRange around it.

ITEM 1–4

The workload contract

YAML

```yaml
      terminationGracePeriodSeconds: 15
    securityContext: {runAsNonRoot: true, seccompProfile: {type: RuntimeDefault}}
    containers:
      - name: api
        image: pinboard-api:1.0
        startupProbe:   {httpGet: {path: /healthz, port: http}, failureThreshold: 30}
        readinessProbe: {httpGet: {path: /readyz,  port: http}, periodSeconds: 5}
        livenessProbe:  {httpGet: {path: /healthz, port: http}, periodSeconds: 10}
        resources: {requests: {cpu: 50m, memory: 32Mi}, limits: {memory: 64Mi}}
        securityContext:
          allowPrivilegeEscalation: false
          readOnlyRootFilesystem: true
          capabilities: {drop: ["ALL"]}
```



Three probes, two resource numbers, one security block. If a Deployment of yours is missing any of them, that is the review comment.

ITEM 5–6

The namespace contract

OBJECT	WHAT IT GUARANTEES
Namespace + PSA labels	A blast radius, and admission-time rejection of anything not `restricted`
ResourceQuota	The team cannot consume more of the cluster than it was given
LimitRange	Containers get sane default requests even when someone forgets
Secret + ConfigMap	Credentials and settings live outside the image — `envFrom`, never `ENV` in a Dockerfile
Service + Ingress	A stable name inside, one documented entry point outside
PDB + HPA	Node drains stay safe; load is absorbed without a human
NetworkPolicy	web → api → db and nothing else; the default is allow-all

02

Packaging & delivery

Twelve YAML files are fine for one environment. Helm and Kustomize are how you survive the second one.

Chart, values, release

- **Chart** — a versioned package of Go-templated manifests
- **Values** — the knobs; `values.yaml` holds the defaults, `-f` and `--set` override them
- **Release** — one install of a chart, with a revision history you can roll back
- Charts live in repos or OCI registries

TEXT

```
charts/pinboard/
├── Chart.yaml          # name, version, appVersion
├── values.yaml         # the defaults
├── templates/
│   ├── deployment.yaml
│   ├── service.yaml
│   ├── ingress.yaml
│   └── _helpers.tpl    # named templates
└── charts/             # vendored dependencies
```

YAML

```yaml
# templates/deployment.yaml
spec:
  replicas: {{ .Values.api.replicas }}
  ...
      image: "{{ .Values.image.repo }}:{{ .Values.image.tag }}"
```

The commands you will actually type

BASH

```
 helm repo add ingress-nginx https://kubernetes.github.io/ingress-nginx
helm repo update

helm install ingress-nginx ingress-nginx/ingress-nginx \
  --namespace ingress-nginx --create-namespace --wait

# render locally and read the YAML — no cluster, no changes
helm template pinboard ./charts/pinboard -f values-dev.yaml

helm upgrade --install pinboard ./charts/pinboard -f values-prod.yaml
helm list -A
helm rollback pinboard 1
helm uninstall pinboard -n pinboard
```

`helm template` is the trust-building command: it turns a chart back into the plain manifests you already know how to read.

KUSTOMIZE

Base and overlays — no templating

TEXT

```
k8s/
├── base/
│   ├── kustomization.yaml
│   ├── 20-api-deployment.yaml
│   └── 40-ingress.yaml
└── overlays/
    ├── dev/
    │   ├── kustomization.yaml
    │   └── hpa-off.yaml
    └── prod/
        └── kustomization.yaml
```

YAML

```yaml
# overlays/dev/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
namespace: pinboard-dev
resources: ["../../base"]
images:   [{name: pinboard-api, newTag: "1.1"}]
replicas: [{name: pinboard-api, count: 1}]
patches:
  - path: hpa-off.yaml
labels:
  - pairs: {app.kubernetes.io/instance: dev}
```

The base is valid YAML you can apply on its own. An overlay only records the differences.

KUSTOMIZE

Render, diff, apply

BASH

```
# render to stdout — nothing touches the cluster
kubectl kustomize k8s/overlays/dev

kubectl apply -k k8s/overlays/dev
kubectl diff  -k k8s/overlays/prod        # what would change?
kubectl delete -k k8s/overlays/dev
```

Reach for Kustomize when…

- The manifests are yours and environments differ by a few fields
- You want zero new syntax and it is already in `kubectl`

Reach for Helm when…

- You are consuming someone else's software
- You need versioned releases, rollback, conditionals and dependencies

DELIVERY

GitOps in one slide

01 — Commit
The desired state is a Git repo — charts or overlays

→

02 — Review
A pull request is the change ticket and the audit trail

→

03 — Agent syncs
Argo CD or **Flux** runs **inside** the cluster and pulls

→

04 — Reconcile
Drift is detected and corrected — or flagged

Nobody runs `kubectl apply` against production. The cluster pulls; CI never needs cluster credentials.

LIVE DEMO · THEN BREAK

helm install ingress-nginx · kubectl apply -k overlays/dev

Install a third-party component from a chart, then roll out our own manifests through a Kustomize overlay — two packaging styles, one cluster, five minutes.

What you'll see · helm template → install → list · overlay diff → apply

Files · demos/session10/, labs/solutions/final/*.yaml

Duration · 8 min

REALITY CHECK

Managed Kubernetes: what changes

LoadBalancer works

A `type: LoadBalancer` Service gets a real cloud load balancer and a public IP. On kind it stays `<pending>` forever — that is why we used an Ingress with port mappings.

StorageClass is real

The default class provisions cloud disks (gp3, Premium SSD, PD). They are zonal — a Pod follows its volume. Set `reclaimPolicy` deliberately.

IAM meets RBAC

A ServiceAccount can be a cloud identity: EKS Pod Identity/IRSA, GKE Workload Identity, AKS workload identity. No static cloud keys in Secrets.

Upgrades are yours

The control plane is managed; node pools are not. Rolling node upgrades drain nodes — which is exactly when your PDB and probes get tested.

AFTER THE COURSE

Where to go next

Certifications

KCNA to consolidate vocabulary · **CKAD** if you ship apps · **CKA** if you run clusters. Both CK* exams are hands-on with the docs open.

Read the source

`kubernetes.io/docs` — Concepts and Tasks are genuinely good. Keep `kubectl explain` and the API reference within reach.

Keep your hands on

killercoda scenarios · **Play with Kubernetes** · or just `kind create cluster` — a throwaway cluster is thirty seconds away.

The fastest way to keep this: take one real service at work and walk it down today's checklist. Everything else is practice.

CAPSTONE · 80 MIN

Deploy Pinboard into `pinboard-prod` — from a blank folder

No copy-paste from the solutions. Start with an empty directory and build the whole stack down the checklist, then run `labs/capstone/verify.sh` and read your scorecard.

The checklist: namespace + PSA labels + ResourceQuota + LimitRange · Secret + ConfigMap · StatefulSet DB with a headless Service and a 1 Gi PVC · API Deployment with three probes, resources, securityContext, PDB and HPA · web Deployment · ClusterIP Services · Ingress on `pinboard.localtest.me` · a smoke-test Job.
Stretch: Kustomize `dev` / `prod` overlays · NetworkPolicy web → api → db · Helm-ify it.

Key takeaways

✓ Production-ready is a checklist

Probes, resources, security context, PDB, HPA, config separation, labels and namespace guardrails — every time, no exceptions.

✓ Package, don't copy

Helm for software you consume, Kustomize overlays for manifests you own. Render before you apply.

✓ Let the cluster pull

GitOps makes Git the desired state and the audit trail; drift gets reconciled instead of discovered.

✓ It all transfers

Managed clusters change LoadBalancers, StorageClasses, IAM and upgrades. The objects you wrote today are the same everywhere.

Zonoth

Thank you

Twenty hours, one small app, and a cluster you can rebuild in thirty seconds. Keep the repo, keep the checklist — and go ship something.